Multi-Pattern Search Engine Using Aho–Corasick Algorithm

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Dharmika P - 2520030012

Darshini G - 2520030167

Parinita T - 2520030562

Team Number: 7

Section: 4

Under the guidance of

Mrs. J. Yamini Devi

Assistant Professor, CSE



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled **"Multi-Pattern Search Engine Using Aho–Corasick Algorithm"** is a record of Bonafide work of **Dharmika P – 2520030012, Darshini G – 2520030167, Parinita T – 2520030562.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Dharmika P - 2520030012

Darshini G - 2520030167

Parinita T - 2520030562

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **"Multi-Pattern Search Engine Using Aho–Corasick Algorithm"** is a bonafide work done and submitted by **Dharmika P – 2520030012, Darshini G – 2520030167, Parinita T – 2520030562.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. J. Yamini Devi, Assistant Professor, CSE which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Dharmika P - 2520030012

Darshini G - 2520030167

Parinita T - 2520030562

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1. ABSTRACT

The Multi-Pattern Search Engine is a data structures and algorithms-based web application developed to efficiently search for multiple patterns within a large collection of text messages. The project focuses on the implementation and visualization of the Aho–Corasick string matching algorithm, which is designed to find multiple keywords simultaneously without repeatedly scanning the same text. The system uses the SMS Spam Collection dataset containing 5,574 messages as the primary dataset for demonstrating multi-pattern searching.

The application allows users to enter multiple search patterns such as “free”, “winner”, “cash”, and “prize” and build an Aho–Corasick automaton from those patterns. The algorithm constructs a Trie to represent the patterns and generates failure links using breadth-first traversal. During the search process, the automaton scans each message character by character and follows appropriate transitions and failure links to identify matching patterns efficiently. The system displays the number of patterns, matched messages, total occurrences, matching positions, and execution time.

An interactive algorithm visualizer is also included to demonstrate Trie construction, failure-link creation, state transitions, and pattern matching step by step. The project provides a practical understanding of an important string-matching algorithm and demonstrates its usefulness for applications such as keyword searching, text filtering, spam detection, and information retrieval.

2. INTRODUCTION

In the modern digital world, large amounts of textual information are generated and processed every day through SMS messages, emails, online documents, search systems, and communication platforms. Searching for specific words or patterns within this information is an important requirement in many computer science applications. When only one pattern needs to be searched, simple string-matching techniques can often be used effectively. However, searching for many patterns separately can require repeated scanning of the same text, which may increase the processing time and reduce the efficiency of the search process. Therefore, an efficient technique is required to identify multiple patterns simultaneously within a collection of text.

The Multi-Pattern Search Engine is a Data Structures and Algorithms-based web application developed to address this requirement using the Aho–Corasick algorithm. The Aho–Corasick algorithm is a multiple-pattern string matching algorithm that allows several patterns to be searched simultaneously in a text. The algorithm combines a Trie data structure with failure links to construct an automaton capable of efficiently processing the input text. Instead of restarting the search whenever a mismatch occurs, the algorithm uses failure links to move to an appropriate state and continue the search. This makes the technique particularly useful when a large number of keywords need to be searched in a large text collection.

The proposed project uses the SMS Spam Collection dataset containing 5,574 messages to demonstrate the practical application of multi-pattern searching. Users can provide multiple patterns, such as “free”, “winner”, “cash”, and “prize”, and build an Aho–Corasick automaton from those patterns. The system searches the messages and identifies the messages in which the selected patterns occur. It also provides information such as the number of patterns searched, the number of matched messages, the total number of occurrences, matching positions, and

execution time. This allows the user to understand not only whether a pattern exists but also how frequently and where it occurs in the dataset.

A major feature of the project is the interactive Algorithm Visualizer. It provides a step-by-step representation of the Aho–Corasick algorithm, including the construction of the Trie, creation of failure links, movement between states, processing of characters, and identification of pattern matches. This visual representation makes the internal working of the algorithm easier to understand compared with observing only the final search results.

The project demonstrates how an important data structure and string-matching algorithm can be applied to a practical text-searching problem. It provides a clear connection between theoretical concepts such as Trie structures, failure links, automata, and pattern matching and their implementation in a web-based application. The system can also serve as a foundation for applications such as keyword filtering, spam detection, text analysis, information retrieval, and content searching.

The remainder of this report presents the related work and literature associated with multi-pattern string searching, followed by the problem statement and objectives of the project. It then describes the hardware and software requirements and explains the implementation of the major modules. The results and discussion section presents the outputs obtained from the system, while the final section summarizes the project and discusses possible future enhancements.

3. LITERATURE SURVEY

The Aho–Corasick algorithm and related multi-pattern searching techniques have been studied and implemented in several research works, programming resources, and information-retrieval systems. These works provide the theoretical foundation and practical understanding required for developing an efficient multi-pattern search engine. The following literature was reviewed to understand Trie structures, failure links, pattern matching, string-searching techniques, and applications of text searching.

3.1 Aho–Corasick String Matching Algorithm

The original work by Alfred V. Aho and Margaret J. Corasick introduced an efficient algorithm for locating occurrences of a finite set of keywords within an input text. The algorithm constructs a finite-state pattern-matching machine using a Trie and failure transitions. This approach allows multiple keywords to be searched during a single scan of the input text. The work provides the fundamental theoretical basis for the Multi-Pattern Search Engine developed in this project.

3.2 CP-Algorithms – Aho–Corasick Algorithm

CP-Algorithms provides an educational implementation and explanation of the Aho–Corasick algorithm for multiple pattern matching. It explains how patterns are inserted into a Trie and how suffix or failure links are constructed to handle mismatches. The resource also discusses the complexity and implementation details of the algorithm. This reference was useful for understanding the algorithmic structure and translating the theoretical concept into an implementable solution.

3.3 GeeksforGeeks – Aho–Corasick Algorithm

GeeksforGeeks presents the Aho–Corasick algorithm as a technique for searching multiple patterns efficiently in a text. Its explanation describes Trie construction, failure links, and the process of traversing the automaton during pattern matching. The implementation examples provide a practical view of how the algorithm can be programmed. This resource helped in understanding the individual stages involved in building and searching an Aho–Corasick automaton.

3.4 GeeksforGeeks – Trie Data Structure

Trie is one of the fundamental data structures used by the Aho–Corasick algorithm to represent a collection of patterns. The GeeksforGeeks Trie resource explains how strings are inserted and searched using character-based nodes. Understanding Trie construction is important because the initial pattern structure of the proposed search engine is created using this concept. This reference was therefore useful for understanding the underlying data structure before implementing failure links and multi-pattern searching.

3.5 CP-Algorithms – String Algorithms

CP-Algorithms provides a collection of string-processing algorithms that demonstrate different approaches to searching and processing textual data. The resource covers several important techniques used in pattern matching and string algorithms. Studying these techniques provides a basis for comparing single-pattern and multi-pattern approaches. This helped establish why an

automaton-based approach such as Aho–Corasick is suitable when several patterns must be searched simultaneously.

3.6 CP-Algorithms – Prefix Function and Pattern Matching

The prefix-function material on CP-Algorithms explains concepts used in efficient string matching and forms a useful comparison with other pattern-searching approaches. Algorithms based on prefix information can avoid unnecessary comparisons while searching for a pattern in text. Studying this approach helped provide a broader understanding of efficient string matching and the differences between single-pattern and multi-pattern searching. The comparison supports the selection of Aho–Corasick for searching multiple keywords simultaneously.

3.7 CP-Algorithms – String Hashing

String hashing is another technique that can be used for comparing and searching strings efficiently. The CP-Algorithms resource explains how strings can be represented using hash values and how these values can be used in pattern-searching applications. Although hashing is different from the Trie and failure-link approach used in this project, studying it provides an understanding of alternative methods for text searching. This comparison helped in identifying the advantages of using an automaton specifically for multiple predefined patterns.

3.8 Stanford Information Retrieval

The Stanford Introduction to Information Retrieval material discusses techniques used to process, search, and retrieve information from large collections of text. Information retrieval systems commonly require efficient methods for identifying relevant terms and matching textual

content. The concepts presented in this resource provide broader application context for keyword and text searching. They are relevant to the proposed system because the project demonstrates multi-pattern searching over a collection of SMS messages.

3.9 Stanford CS97SI – Introduction to Competitive Programming

The Stanford CS97SI course material provides educational coverage of important algorithmic techniques and data structures used for efficient problem solving. String algorithms and data structures such as Tries are relevant to the implementation of efficient pattern-searching systems. Studying these concepts helped strengthen the algorithmic understanding required for implementing the proposed application. The resource also provides useful background for analysing algorithm efficiency and selecting appropriate data structures.

3.10 Comparison of Existing Approaches with the Proposed System

The reviewed works demonstrate that efficient text searching can be achieved through different data structures and algorithms, including Tries, prefix-based algorithms, hashing techniques, and automaton-based pattern matching. However, the proposed Multi-Pattern Search Engine focuses specifically on the practical implementation and visualization of the Aho–Corasick algorithm. In addition to searching multiple patterns in the SMS dataset, the proposed application provides an interactive visualization of Trie construction, failure-link creation, state transitions, and pattern matching. Therefore, the project combines algorithmic implementation with an educational visual interface to make multi-pattern searching easier to understand and demonstrate.

4. Problem Statement and Objectives

4.1 Problem Statement

Efficiently searching for multiple patterns simultaneously in a large collection of text messages is challenging when traditional single-pattern searching methods require repeated scans of the same text.

4.2 Problem Description

Searching multiple keywords independently can increase processing time and lead to repeated comparisons across the same text data. An efficient multi-pattern searching approach is therefore required to identify multiple patterns quickly and accurately.

4.3 Objectives

The main purpose of this project is to develop a Multi-Pattern Search Engine that efficiently searches multiple keywords in a collection of SMS messages using the Aho–Corasick algorithm. The project aims to demonstrate how an advanced string-matching algorithm can be applied to a practical text-searching problem while providing an interactive interface for understanding its operation.

The first objective is to implement the Aho–Corasick algorithm using a Trie-based structure and failure links so that multiple patterns can be searched simultaneously without repeatedly scanning the complete text for every pattern.

The second objective is to provide a simple web-based interface through which users can enter multiple patterns and search them across the SMS dataset. The system should

display the patterns searched, matched messages, total occurrences, matching positions, and search execution time.

The third objective is to develop an interactive Algorithm Visualizer that demonstrates the major stages of the Aho–Corasick algorithm, including Trie construction, failure-link creation, state transitions, character processing, and pattern detection.

The fourth objective is to provide a practical demonstration of efficient multi-pattern searching using the SMS Spam Collection dataset containing 5,574 messages. This helps connect theoretical concepts from Data Structures and Algorithms with a real-world text-processing application.

The final objective is to create an educational and user-friendly system that improves understanding of multi-pattern string matching and demonstrates how the Aho–Corasick algorithm can be applied to applications such as keyword searching, text filtering, spam detection, and information retrieval.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Computer or laptop with a minimum of 4 GB RAM.
- Dual-core or higher processor for running the application and processing the dataset.
- Minimum 2 GB available storage for project files, dataset, and dependencies.
- Keyboard, mouse, and display for interacting with the application.
- Stable internet connection for installing dependencies and development purposes.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Visual Studio Code (VS Code): Used as the primary development environment.
- Node.js: Required to run the React development environment.
- npm (Node Package Manager): Used for installing and managing project dependencies.
- React: Used for developing the web application interface.
- JavaScript: Used for implementing the application logic and Aho–Corasick functionality.
- HTML and CSS: Used for structuring and styling the application.
- SMS Spam Collection Dataset: Used for testing and demonstrating multi-pattern searching.
- Web Browser: Google Chrome or Microsoft Edge for running and demonstrating the application.

6. IMPLEMENTATION

The Multi-Pattern Search Engine is implemented as a web-based application using React and JavaScript. The main implementation is divided into modules that handle pattern processing, Aho–Corasick automaton construction, multi-pattern searching, and algorithm visualization. These modules work together to allow users to enter multiple patterns, build the automaton, search the SMS dataset, and observe the algorithm step by step.

6.1 Pattern Input and Trie Construction

The first module accepts multiple patterns from the user and stores them in a Trie data structure. Each character of a pattern is represented as a node in the Trie. Common prefixes of different patterns are stored only once, which makes the structure suitable for multi-pattern matching.

src > components > Analytics.jsx > ...

```jsx
import { useEffect, useState } from 'react'
import * as api from '../services/api'
import { Head } from './Visualizer'
export default function Analytics({ result }) {
  const [a, setA] = useState(null)
  useEffect(() => { if (result) api.getAnalytics().then(setA); else setA(null) }, [result])
  if (!a) return <section><Head title="Search Analytics" sub="Pattern frequency and search statistics." /><div className="glass pad mut
  const max = Math.max(...a.frequency.map((f) => f.count), 1)
  const total = a.spamMatches + a.hamMatches || 1
  const top = [...a.frequency].sort((x, y) => y.count - x.count)[0]
  return (
    <section>
      <Head title="Search Analytics" sub="Pattern frequency and search statistics for the last search." />
      <div className="cards">
        <div className="glass stat"><small>Most frequent pattern</small><strong>{top.pattern}</strong></div>
        <div className="glass stat"><small>Average position</small><strong>{a.avgPosition}</strong></div>
        <div className="glass stat"><small>Occurrences per matched message</small><strong>{(result.totalOccurrences / (result.matchedMe
        <div className="glass stat"><small>Execution time</small><strong>{result.executionMs} ms</strong></div>
      </div>
      <div className="two">
        <div className="glass"><h3>Pattern frequency</h3>
          {a.frequency.map((f) => (
            <div key={f.pattern}><span>{f.pattern}</span>
              <div><i style={{ width: (f.count / max) * 100 + '%' }} /></div><b>{f.count}</b></div>
          ))}
        </div>
        <div className="glass"><h3>Spam vs ham among matched messages</h3>
          <div><i style={{ width: (a.spamMatches / total) * 100 + '%' }} /></div>
          <p><span>spam {a.spamMatches}</span><span>ham {a.hamMatches}</span></p>
        </div>
      </div>
    </section>
  )
}
```

The TrieNode class stores the child nodes, failure link, and output patterns. The buildTrie() function inserts each input pattern into the Trie and marks the end of every pattern. This forms the basic structure required for constructing the Aho–Corasick automaton.

6.2 Failure Link Construction

The second module constructs failure links for the Trie using Breadth-First Search. A failure link determines the next appropriate state when the current character does not have a matching transition. This allows the algorithm to continue searching without restarting from the beginning of the text.

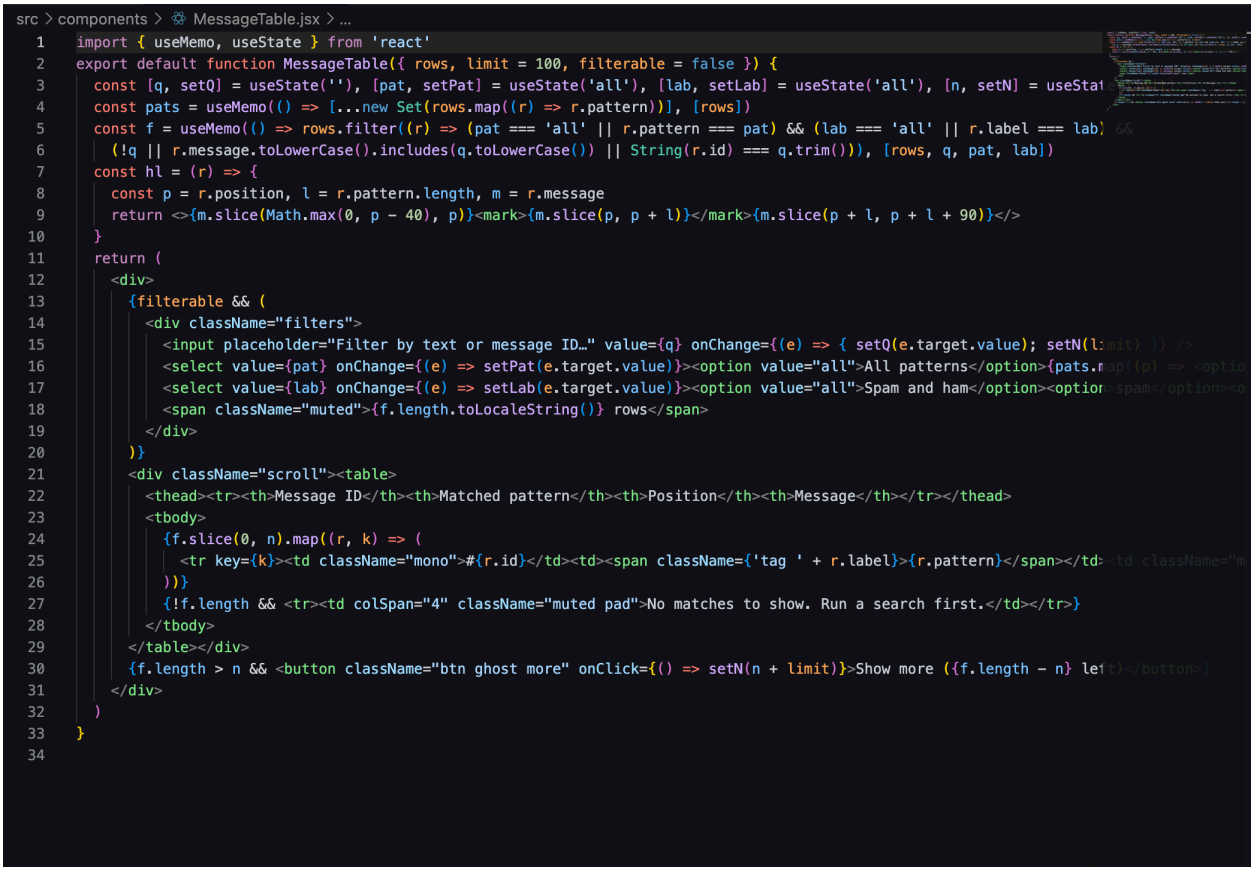


The function starts by assigning the root as the failure state of the first-level nodes. It then processes the remaining nodes using a queue. For every character transition, the appropriate

failure state is found and assigned. Output patterns from the failure state are also inherited, allowing the system to detect multiple patterns efficiently.

6.3 Multi-Pattern Search

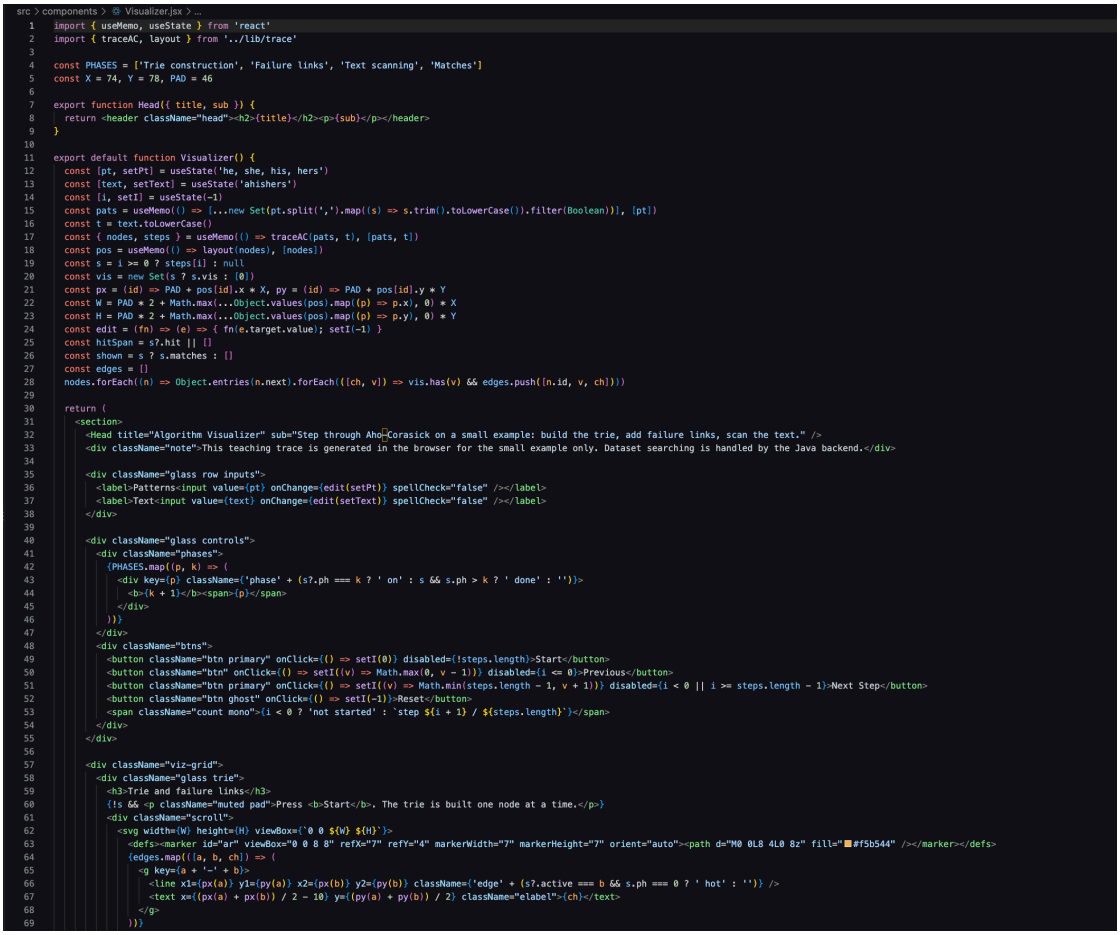
The third module performs the actual search through the SMS dataset. After the Trie and failure links have been constructed, each message is processed character by character. When a pattern is detected, the pattern name, message index, and matching position are recorded.

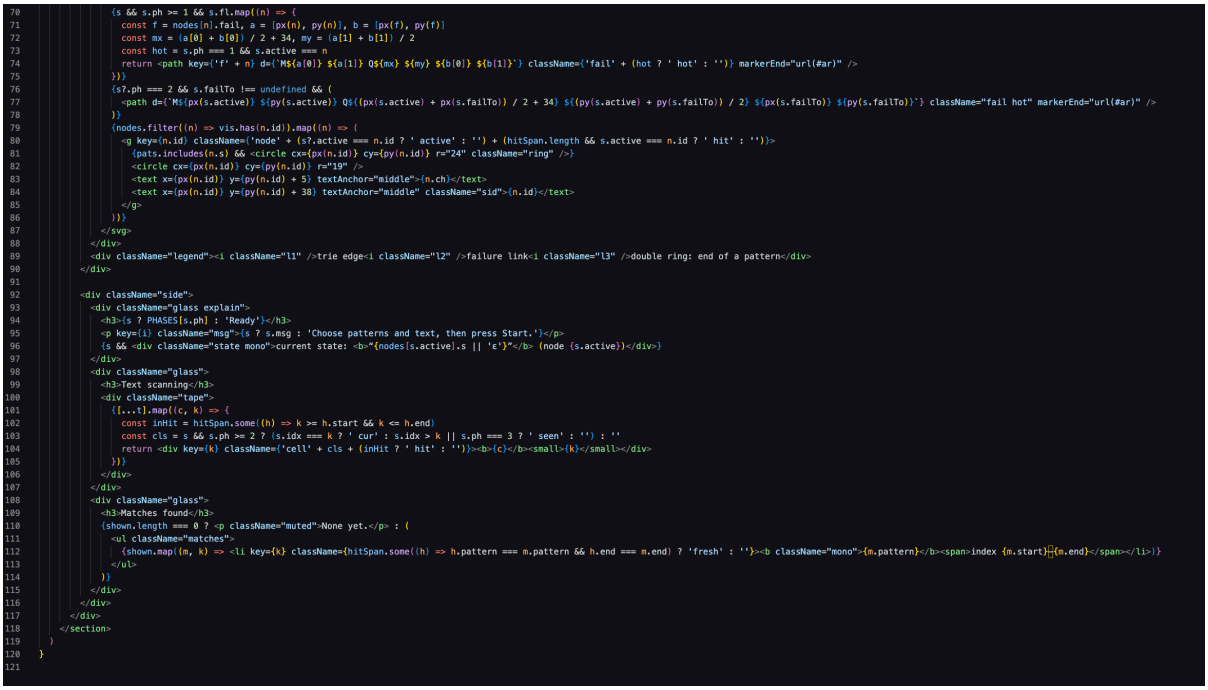


The search function follows Trie transitions whenever the current character matches. If a transition is unavailable, the algorithm follows failure links until a valid transition is found or the root is reached. Whenever the current state contains output patterns, the corresponding matches are stored with their positions. This avoids performing a separate complete search for every pattern.

6.4 Algorithm Visualization Module

The fourth module provides an interactive visualization of the Aho–Corasick algorithm. It represents Trie nodes and their relationships and allows the user to observe the construction and searching process step by step. The visualization helps users understand how patterns are inserted, how failure links are created, and how the automaton moves between states while





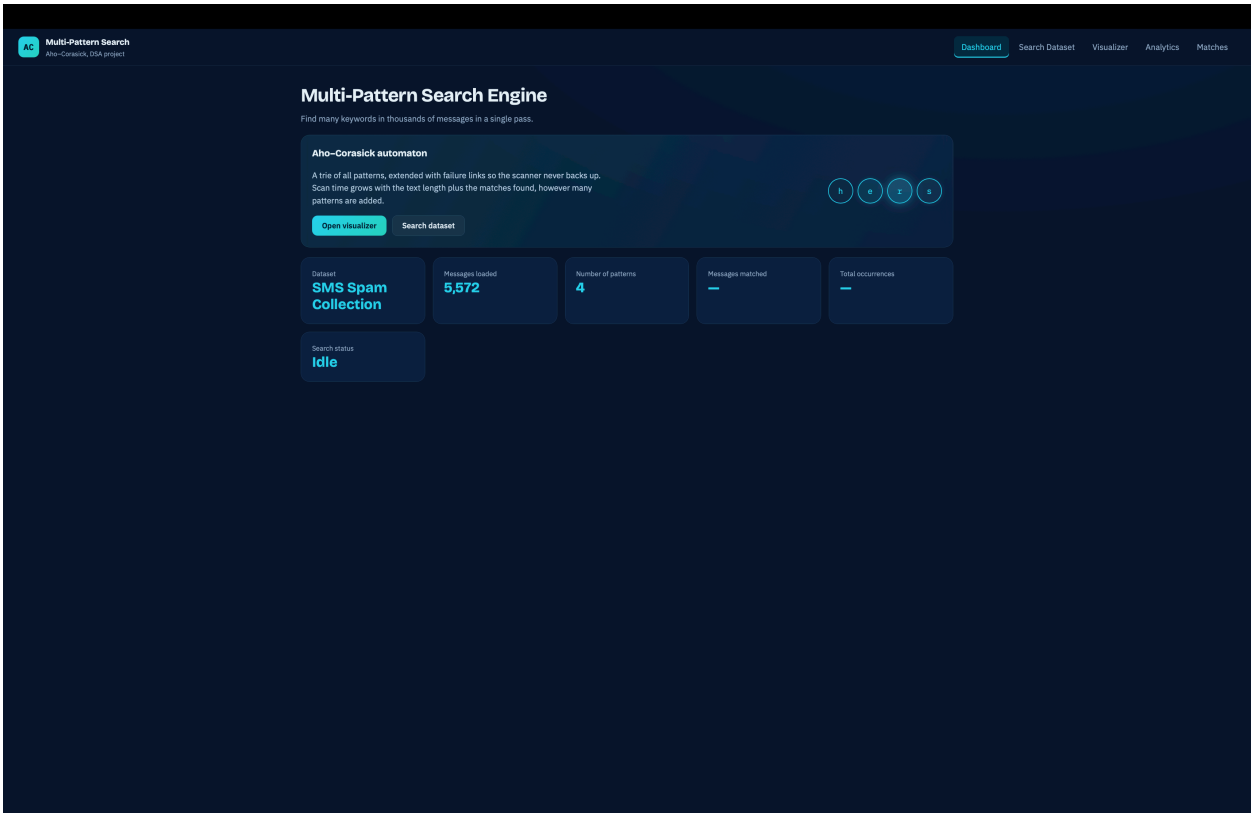
```
          {s && s.ph >= 1 && s.fl.map((n) => {
            const f = nodes[n].fail, a = [px(n), py(n)], b = [px(f), py(f)]
            const mx = (a[0] + b[0]) / 2 + 34, my = (a[1] + b[1]) / 2
            const hot = s.ph === 1 && s.active === n
            return <path key={'f' + n} d={`M${a[0]} ${a[1]} Q${mx} ${my} ${b[0]} ${b[1]}`} className={'fail' + (hot ? ' hot' : '')} markerEnd="url(#ar)" />
          })}
          {s?.ph === 2 && s.failTo !== undefined && (
            <path d={`M${px(s.active)} ${py(s.active)} Q${(px(s.active) + px(s.failTo)) / 2 + 34} ${(py(s.active) + py(s.failTo)) / 2} ${px(s.failTo)} ${py(s.failTo)}`} className="fail hot" markerEnd="url(#ar)" />
          )}
          {nodes.filter((n) => vis.has(n.id)).map((n) => (
            <g key={n.id} className={'node' + (s?.active === n.id ? ' active' : '') + (hitSpan.length && s.active === n.id ? ' hit' : '')}>
              {pats.includes(n.s) && <circle cx={px(n.id)} cy={py(n.id)} r="24" className="ring" />}
              <circle cx={px(n.id)} cy={py(n.id)} r="19" />
              <text x={px(n.id)} y={py(n.id) + 5} textAnchor="middle">{n.ch}</text>
              <text x={px(n.id)} y={py(n.id) + 38} textAnchor="middle" className="sid">{n.id}</text>
            </g>
          ))}
        </svg>
      </div>
      <div className="legend"><i className="l1" />trie edge<i className="l2" />failure link<i className="l3" />double ring: end of a pattern</div>
    </div>

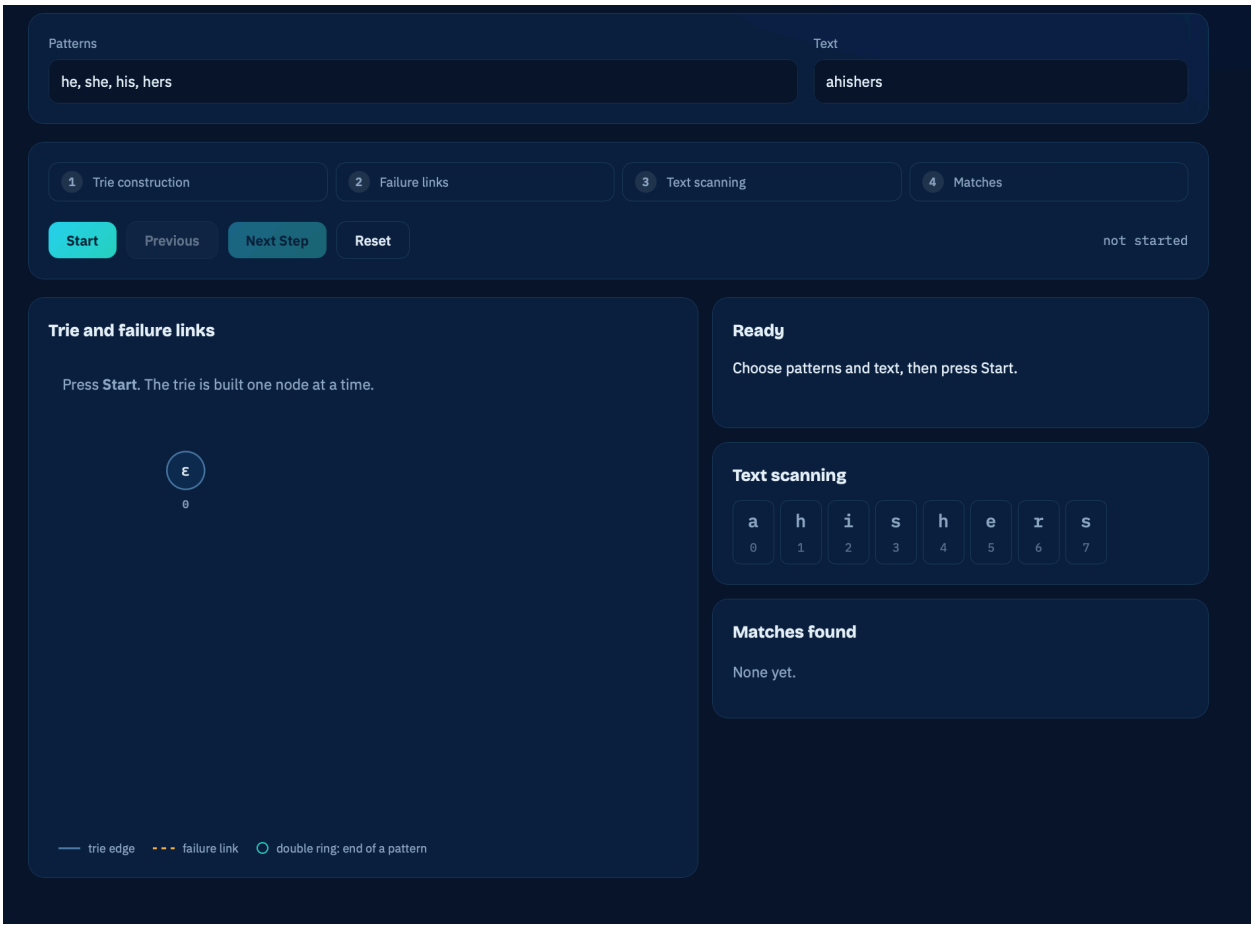
    <div className="side">
      <div className="glass explain">
        <h3>{s ? PHASES[s.ph] : 'Ready'}</h3>
        <p key={i} className="msg">{s ? s.msg : 'Choose patterns and text, then press Start.'}</p>
        {s && <div className="state mono">current state: <b>"{nodes[s.active].s || 'ε'}"</b> (node {s.active})</div>}
      </div>
      <div className="glass">
        <h3>Text scanning</h3>
        <div className="tape">
          {[...t].map((c, k) => {
            const inHit = hitSpan.some((h) => k >= h.start && k <= h.end)
            const cls = s && s.ph >= 2 ? (s.idx === k ? ' cur' : s.idx > k || s.ph === 3 ? ' seen' : '') : ''
            return <div key={k} className={'cell' + cls + (inHit ? ' hit' : '')}><b>{c}</b><small>{k}</small></div>
          })}
        </div>
      </div>
      <div className="glass">
        <h3>Matches found</h3>
        {shown.length === 0 ? <p className="muted">None yet.</p> : (
          <ul className="matches">
            {shown.map((m, k) => <li key={k} ><b>{m.pattern}</b><span>index {m.start}–{m.end}</span></li>)}
          </ul>
        )}
      </div>
    </div>
  </section>
  )
}
```

This module generates individual visualization steps that can be displayed by the React interface. The user can observe the stages of pattern insertion, failure-link construction, character processing, and completion of the search. The visualization makes the internal working of the algorithm easier to understand and provides an important educational component of the project.

Together, these modules implement the main functionality of the Multi-Pattern Search Engine. The Trie module organizes the input patterns, the failure-link module constructs the Aho–Corasick automaton, the search module identifies multiple patterns in the SMS dataset, and the visualization module demonstrates the algorithm's operation interactively.

7. RESULTS & DISCUSSION





8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Multi-Pattern Search Engine successfully demonstrates the application of the Aho–Corasick algorithm for searching multiple patterns within a collection of text messages. The project achieved its primary objective of providing an efficient approach for identifying several keywords without performing a separate complete search for every pattern. By using a Trie and failure links, the system can process multiple patterns and identify their occurrences in the SMS dataset.

The project also provides a user-friendly web interface through which users can enter multiple patterns, build the Aho–Corasick automaton, and search the dataset. The system displays useful search information such as matched messages, total occurrences, matching positions, and execution time. The interactive Algorithm Visualizer further helps users understand the construction and operation of the Trie, failure links, state transitions, and pattern matching process.

Overall, the project successfully connects theoretical concepts from Data Structures and Algorithms with a practical text-searching application. It demonstrates how the Aho–Corasick algorithm can be used for efficient multi-pattern searching and provides an educational platform for understanding an important string-matching technique.

8.2 Future Scope

The project can be enhanced in the future by supporting larger and more diverse datasets so that the performance of the Aho–Corasick algorithm can be evaluated on different types and sizes of text collections. The system can also be extended to allow users to upload their own text files or datasets instead of relying only on the existing SMS dataset.

In the near future, the search interface can be improved by providing advanced filtering options, case-sensitive or case-insensitive searching, pattern grouping, and more detailed search statistics. The visualization can also be enhanced with animated Trie construction, animated failure-link creation, highlighted matching characters, and controls for moving forward and backward through individual algorithm steps.

In the medium term, the application can be extended to support real-time text streams such as incoming messages, documents, or other continuously generated text. Additional algorithms such as KMP, Rabin–Karp, and other pattern-matching techniques could also be incorporated so that users can compare their performance with the Aho–Corasick algorithm.

In the long term, the system could be developed into a larger text-analysis platform capable of handling very large datasets and practical applications such as spam filtering, keyword monitoring, content filtering, document searching, and information retrieval. Performance optimization, database integration, and deployment as a scalable web application could further improve its practical usability.

9. REFERENCES

- V. Aho and M. J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search," Communications of the ACM, 1975. Available at: https://dl.acm.org/doi/10.1145/360825.360855
- CP-Algorithms, "Aho-Corasick Algorithm," Available at: https://cp-algorithms.com/string/aho_corasick.html
- GeeksforGeeks, "Aho-Corasick Algorithm for Pattern Searching," Available at: https://www.geeksforgeeks.org/aho-corasick-algorithm-pattern-searching/
- GeeksforGeeks, "Trie | Insert and Search," Available at: https://www.geeksforgeeks.org/trie-insert-and-search/
- CP-Algorithms, "String Algorithms," Available at: https://cp-algorithms.com/string/
- CP-Algorithms, "Prefix Function – Knuth-Morris-Pratt," Available at: https://cp-algorithms.com/string/prefix-function.html
- CP-Algorithms, "String Hashing," Available at: https://cp-algorithms.com/string/string-hashing.html
- Stanford University, "Introduction to Information Retrieval," Available at: https://nlp.stanford.edu/IR-book/
- Stanford University, "CS97SI: Introduction to Competitive Programming," Available at: https://web.stanford.edu/class/cs97si/